

I&C Laundry | Proposed ETL Pipeline

AI-Assisted Business Analytics and Decision Support System for I&C Laundry Shop

1. Pipeline Overview

The proposed Daily Branch and Service Reporting ETL Pipeline prepares completed-day order data for business analytics across I&C Laundry's three branches. It extracts records from the existing shared Supabase PostgreSQL database, cleans and summarizes them using SQL, and loads reporting summary tables on a daily Supabase Cron schedule.

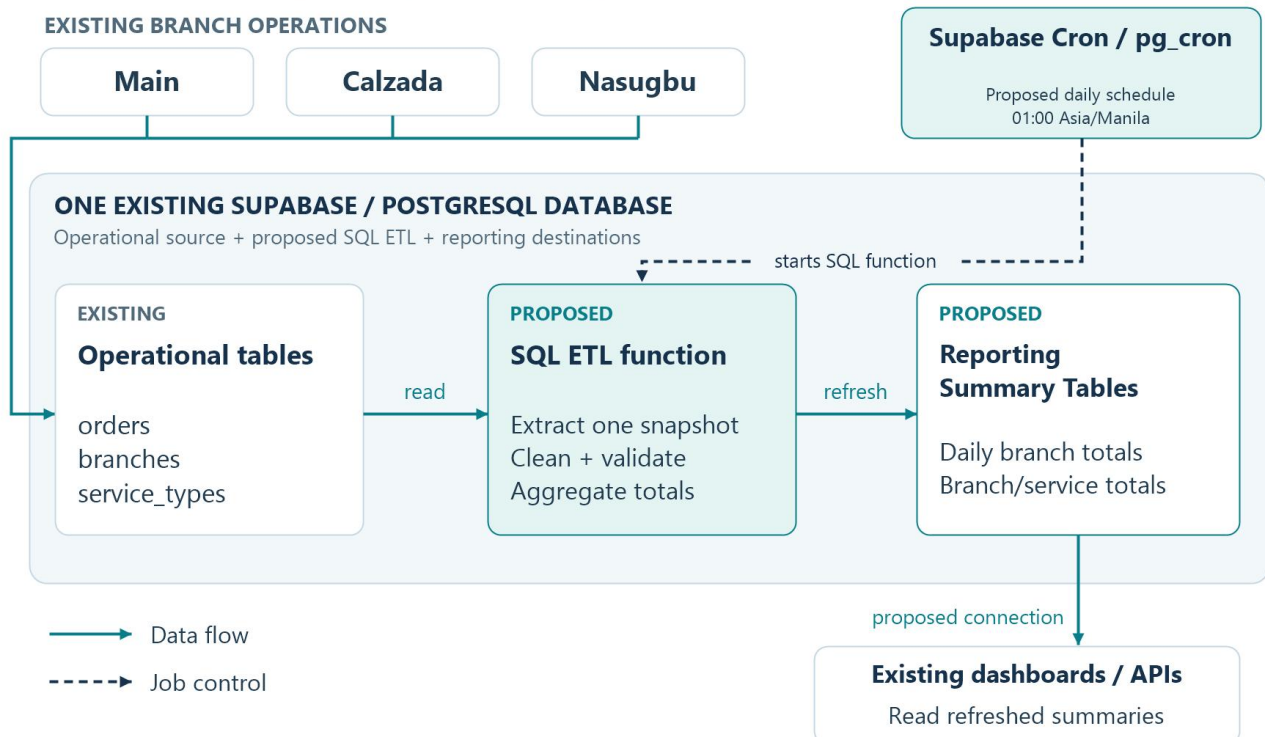
The pipeline addresses repeated report calculations and inconsistent grouping of dates, branches, and services. Its outputs support sales monitoring, branch workload comparisons, service-demand analysis, and daily customer activity. Prepared daily totals can also supply historical inputs to the system's forecasting features. Forecast generation itself is outside this ETL pipeline.

Design status: The source tables are based on the project's SQL schema and migrations. The scheduled function, temporary stage, summary tables, and dashboard connection described here are proposed; this document does not claim that they are already deployed.

2. Pipeline Architecture

PROPOSED ETL ARCHITECTURE

I&C Laundry | SQL batch processing inside the existing database



The ETL function and summary tables are proposed additions to the existing database.

Figure 1. Proposed architecture using the four technologies from the earlier ETL stack.

Supabase Cron supplies the trigger; SQL performs the processing; PostgreSQL holds both operational data and prepared reporting tables. Application access to the summaries would use the existing authenticated backend.

3. Pipeline Metadata

Table 1. Pipeline definition, schedule, dependencies, configurations, and connections.

Metadata	Description
Pipeline name	I&C Laundry Daily Branch and Service Reporting ETL Pipeline
Purpose	Prepare consistent historical totals for sales, order demand, branch workload, service performance, and daily customer activity.
Technologies / tools	PostgreSQL / Supabase; Supabase Cron / pg_cron; SQL; Reporting Summary Tables.
Data source	Existing public.orders, public.branches, and public.service_types in one shared Supabase database. Customer activity uses orders.customer_id, which references public.customers.id.
Destination / storage	Proposed analytics.daily_branch_summary and analytics.daily_service_summary tables in the same PostgreSQL database.
Schedule	Daily at 01:00 Asia/Manila (UTC+08:00). With cron.timezone set to UTC/GMT, use cron expression 0 17 * * *. Verify the scheduler timezone before enabling the job.
Run configuration	Job name: ic_laundry_daily_reports. Function: analytics.refresh_daily_reports(). One run at a time, protected by a transaction-scoped advisory lock.
Data window / refresh	Full historical rebuild for order creation dates earlier than the local date at run start. Rebuilding previous dates captures late payments, cancellations, and corrections. Current-day transactions remain available from operational screens.
Dependencies	Required source columns and foreign keys; enabled pg_cron extension; deployed SQL function and target tables; a database role permitted to read all three branches and update summaries.
Processing settings	Business timezone: Asia/Manila. Currency: PHP. Exact NUMERIC arithmetic. Capture one run timestamp and one source snapshot. Proposed statement timeout: 5 minutes; revisit after measuring real data volume.
Connections	pg_cron → SQL function: in-database invocation. SQL → public tables: SELECT/JOIN. SQL → analytics tables: transactional writes. Backend → summaries: authorized database reads for reports. No external ETL server is required.
Pipeline steps	1 Read saved orders → 2 Extract a snapshot → 3 Validate and clean → 4 Aggregate → 5 Publish both summaries → 6 Read results in reports.
Failure handling	Validation errors raise an exception and roll back the refresh; the previous published summaries remain available. Review cron.job_run_details, correct the cause, and rerun. No automatic retry is assumed.
Publication / access	Publish both tables in one transaction. Replace prior summary rows rather than appending duplicate totals. Display refreshed_at as the report's data timestamp; retain the backend's existing authorization and branch restrictions.

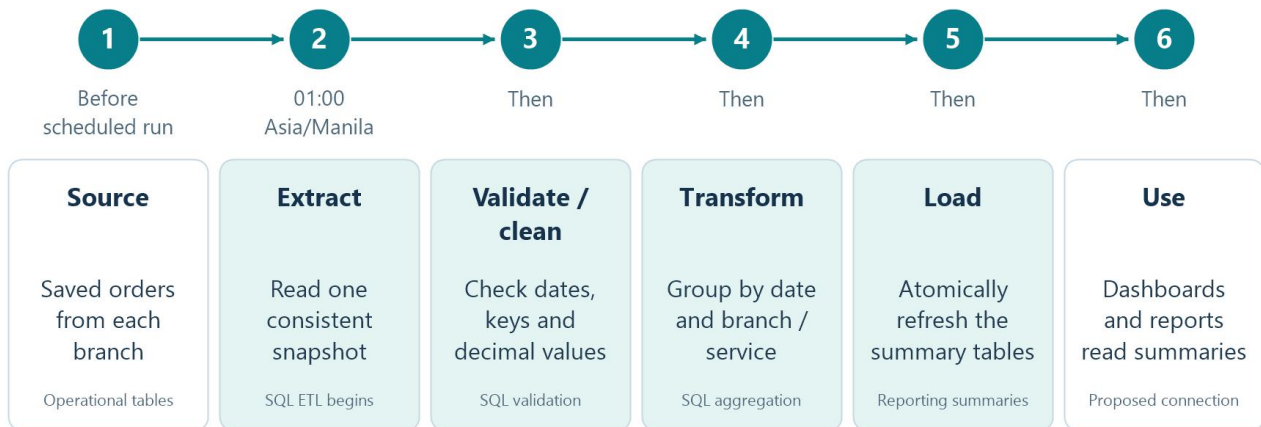
All scheduling and processing values above are proposed configuration choices. Supabase Cron documents SQL-function scheduling and run-history monitoring; the database timezone used for business dates is distinct from the scheduler's timezone. [1]

4. Data Lineage

The timeline shows the sequence of one scheduled run. Each stage consumes the preceding stage's output; it is not a measurement of execution time.

DATA LINEAGE TIMELINE

Proposed daily run | Follow the numbered stages from left to right.



If validation fails: stop the job and retain the previous summary tables.

This timeline shows the order of operations, not measured runtime. No stage durations are assumed.

Figure 2. Data-lineage timeline for the proposed daily ETL run.

Table 2. Inputs, transformations, and outputs at each stage.

Stage	Input	Process / transformation	Output
1. Source	Orders from all three branches; branch and service lookups.	Operational transactions save order IDs, payment totals, status, weight, and related IDs in PostgreSQL.	Committed operational records.
2. Extract	public.orders, branches, and service_types.	Read one joined snapshot into a temporary SQL stage. Retain invalid rows for validation, including null timestamps. Capture the run timestamp once.	One staged row per order.
3. Validate / clean	Staged source rows.	Check unique IDs, branch/service references, timestamps, statuses, and nonnegative values. Reject null timestamps before date filtering. Derive report_date in Asia/Manila.	Validated, consistently dated order rows.
4. Transform	Validated rows.	Keep dates through yesterday; exclude cancelled orders. Group by branch/day and service/day. Count orders, sum weight/value/payments, and count distinct customers for branch/day only.	Two sets of grouped summary rows.
5. Load	Branch and service summaries.	Validate reconciliation totals; replace both reporting tables in one transaction and set refreshed_at. Any error rolls back the replacement.	Published reporting summary tables.
6. Use	Last successful summaries.	Backend reads by date, branch, or service for dashboards/reports. Show the refresh timestamp; fill missing chart dates with zero.	Consistent historical report data.

A duplicate order ID or an unresolved non-null branch/service reference stops the batch. Customer references rely on the source foreign key. A missing branch is an error; a genuinely null service or customer ID is handled using the rules in Section 5.

5. Schema Metadata

5.1 Operational source tables — selected columns

Only fields used by this pipeline are listed. PK means primary key; FK means foreign key. These definitions come from the repository's schema and migrations, not a live-database inspection. The three branches share these tables.

Table 3. *public.orders* — one row per laundry order.

Column	Data type	Key / meaning
id	UUID	PK; stable order identifier and staging uniqueness key.
branch_id	UUID	Nullable FK → public.branches.id; must resolve for reporting.
customer_id	UUID	Nullable FK → public.customers.id; used for distinct-customer counts.
service_type_id	UUID	Nullable FK → public.service_types.id; null means Unspecified.
created_at	TIMESTAMPTZ	Source creation timestamp; required to derive the report date.
status	TEXT	Order state: received, on_process, ready, released, or cancelled.
payment_status	TEXT	Payment state: unpaid, partial, or paid.
weight_kg	NUMERIC(10,2)	Recorded order weight; nonnegative.
total_price	NUMERIC(10,2)	Final order value after any applied loyalty discount.
amount_paid	NUMERIC(10,2)	Cumulative recorded payment for the order; normally NOT NULL.

Table 4. *Reference tables* — fields used for joins and labels.

Table / column	Data type	Key / meaning
public.branches.id	UUID	PK in declared schema; uniquely identifies a branch.
public.branches.name	TEXT	Branch label; declared NOT NULL and unique.
public.branches.code	TEXT	Unique branch code; may be null.
public.service_types.id	UUID	PK; identifies a recorded service category.
public.service_types.name	TEXT	NOT NULL; service label. Preserve historical lookup records.
public.customers.id	UUID	PK referenced by orders.customer_id. Names, phone numbers, and emails are not needed for these summaries.

Temporary stage: pg_temp.etl_orders_stage copies the selected orders columns above, joined to lookup records for key validation, and adds report_date DATE. It enforces one row per order ID and is dropped at transaction end. It is transient working data, not an additional reporting destination.

Source basis: supabase_schema.sql; 20260906_multi_branch_reporting.sql; 20260908_three_stage_order_eta.sql; 20260909_cross_branch_loyalty_program.sql. Existing analytics behavior was checked in frontend/src/pages/Analytics.jsx.

5.2 Proposed reporting summary tables

All target columns are NOT NULL unless stated otherwise. No individual customer contact details are stored in these summaries.

Table 5. *analytics.daily_branch_summary* — one row per date and branch.

Column	Data type	Key / calculation
report_date	DATE	Composite PK, part 1; local order creation date.
branch_id	UUID	Composite PK, part 2; FK → public.branches.id.
order_count	BIGINT	COUNT(*) for non-cancelled orders in the group.
released_order_count	BIGINT	Count of those orders whose current status is released.
unique_customer_count	BIGINT	COUNT(DISTINCT customer_id); null customer IDs excluded.
total_weight_kg	NUMERIC(14,2)	SUM(weight_kg).
order_value_php	NUMERIC(14,2)	SUM(total_price); use recorded final values.
recorded_payments_php	NUMERIC(14,2)	SUM(normalized amount_paid), attributed to report_date.
refreshed_at	TIMESTAMPTZ	Timestamp captured at the start of the successful run.

Table 6. *analytics.daily_service_summary* — one row per date, branch, and service key.

Column	Data type	Key / calculation
report_date	DATE	Composite PK, part 1; local order creation date.
branch_id	UUID	Composite PK, part 2; FK → public.branches.id.
service_key	TEXT	Composite PK, part 3; service UUID as text, or unspecified.
service_type_id	UUID	Nullable FK → public.service_types.id; null for Unspecified.
order_count	BIGINT	COUNT(*) for the non-cancelled orders in this group.
total_weight_kg	NUMERIC(14,2)	SUM(weight_kg).
order_value_php	NUMERIC(14,2)	SUM(total_price).
recorded_payments_php	NUMERIC(14,2)	SUM(normalized amount_paid), attributed to report_date.
refreshed_at	TIMESTAMPTZ	Same run timestamp as the branch summary.

Constraints and relationships: Each branch has many source orders and summary rows; each service has many orders and service-summary rows. Each customer can have many orders. The target composite keys prevent duplicate groups. Enforce service_key = COALESCE(service_type_id::text, 'unspecified'); use nonnegative checks on counts and totals. Reject numeric overflow rather than rounding away an error.

5.3 Metric and relationship rules

Dates and recorded payments. `report_date` is `(created_at AT TIME ZONE 'Asia/Manila')::date`.

`recorded_payments_php` is the sum of cumulative order payments assigned to that order date, including partial payments. It is not a payment-date cash-flow total. A later balance payment updates the original order date's summary during the next full refresh.

Legacy and status handling. Use `amount_paid` when present; only for legacy null values use `total_price` when `payment_status` is paid, otherwise zero. Preserve final order prices instead of recalculating from current rates. Cancelled orders stay in the operational data but are excluded from report totals; cancellation alone does not imply a refund. `released_order_count` groups orders by creation date, not release date.

Grouping and reconciliation. A null service stays in the Unspecified group, so summing service counts, weights, values, and recorded payments for a branch/date must match the branch summary. Store only groups with qualifying orders; report charts can fill date gaps with zero. Do not sum `unique_customer_count` across dates or branches to infer unique people: the same customer may appear in several groups. Recalculate a wider distinct count from source customer IDs when needed.

Job metadata. Scheduling definitions and outcomes remain in the extension-managed `cron.job` and `cron.job_run_details` tables. They are monitoring records, not additional business reporting tables. Failed validation raises an exception so the job history records failure. [1]

6. Technology Justification

PostgreSQL / Supabase

Selected because the current system already stores relational operational data in Supabase PostgreSQL. The proposed pipeline can read source tables and store results within the same database, using keys and transactions to preserve consistency. Large analytical jobs can compete with live transactions, so this design proposes an off-peak schedule and a measured runtime limit. [2]

Supabase Cron / pg_cron

Selected to start the SQL refresh on a recurring daily schedule and record job outcomes inside PostgreSQL. This fits a short database-processing job and avoids introducing a separate scheduler service. The schedule and job history can be managed through Supabase. [1]

SQL

Selected for extraction, joins, validation, date conversion, grouping, and loading. SQL operates directly on the existing relational data and keeps calculation rules in one scheduled database function. The function makes the proposed stages repeatable and reviewable. [3]

Reporting Summary Tables

Selected as the destination for prepared branch and service totals. They are ordinary PostgreSQL tables, not a separate software product. They reduce repeated dashboard aggregation while making the reporting grain explicit. Their figures reflect the latest successful scheduled refresh, so reports must show `refreshed_at`.

Technology sources

[1] Supabase. (n.d.). Cron; Quickstart. <https://supabase.com/docs/guides/cron> and <https://supabase.com/docs/guides/cron/quickstart>

[2] Supabase. (n.d.). Database overview. <https://supabase.com/docs/guides/database/overview>

[3] PostgreSQL Global Development Group. (n.d.). The SQL Language. <https://www.postgresql.org/docs/current/sql.html>